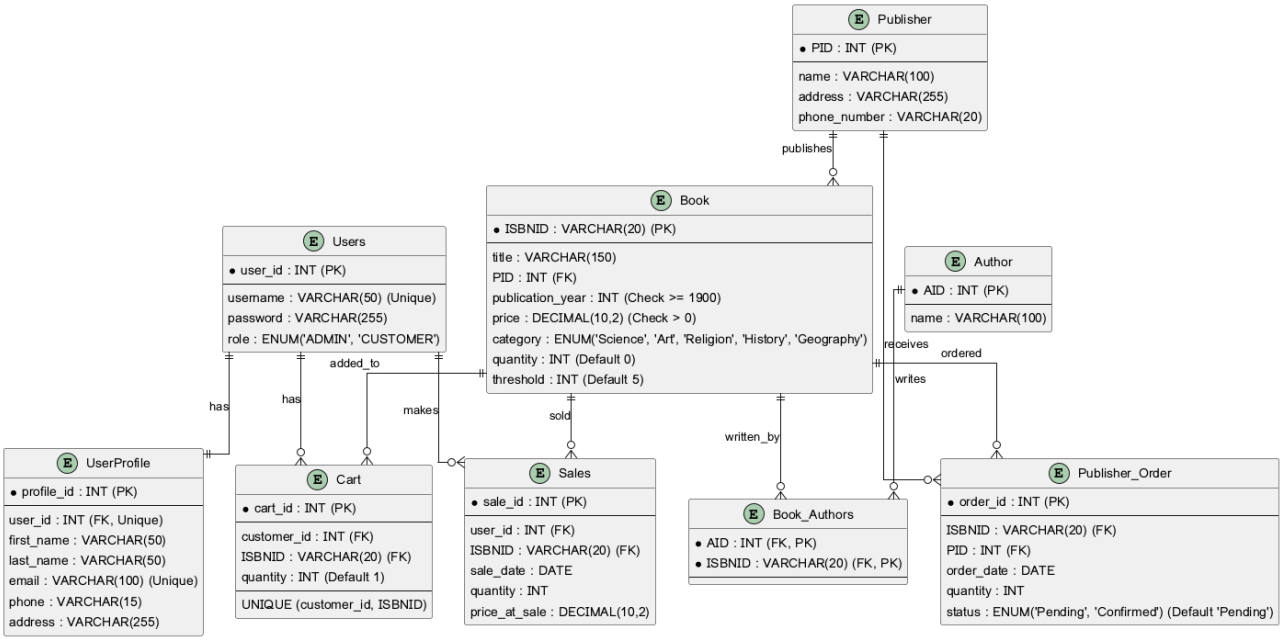
Database Final Project Report: Order Processing System

1. Data Modeling

The foundation of the **Order Processing System** is a robust relational database designed to manage an online bookstore's operations. The model focuses on maintaining data integrity through normalized tables, primary/foreign key relationships, and automated triggers.

1.1 Entity-Relationship Diagram (ERD)

The ERD below illustrates the complete set of entities and their relationships. It captures the 1:1 relationship between **Users** and **UserProfile**, the M:N relationship between **Book** and **Author** (resolved via **Book_Authors**), and the various transactional relationships for **Sales**, **Cart**, and **Publisher_Order**.



1.2 Relational Schema

The following schema is a 1:1 mapping of the implemented SQL database. It includes all attributes, data types, and constraints (Primary Keys, Foreign Keys, Unique, Check, and Enum).

Table Name	Attributes	Constraints	Description
Users	user_id (INT), username (VARCHAR), password (VARCHAR), role (ENUM)	PK: user_id, Unique: username	Stores authentication data and user roles ('ADMIN', 'CUSTOMER').
UserProfile	profile_id (INT), user_id (INT), first_name (VARCHAR), last_name (VARCHAR), email (VARCHAR), phone (VARCHAR), address (VARCHAR)	PK: profile_id, FK: user_id (Unique)	Stores personal details; linked 1:1 with **Users**.
Publisher	PID (INT), name (VARCHAR), address (VARCHAR), phone_number (VARCHAR)	PK: PID	Stores book supplier information.
Book	ISBNID (VARCHAR), title (VARCHAR), PID (INT), publication_year (INT), price (DECIMAL), category (ENUM), quantity (INT), threshold (INT)	PK: ISBNID, FK: PID, Check: pub_year >= 1900, price > 0	Core inventory table. Categories: 'Science', 'Art', 'Religion', 'History', 'Geography'.
Author	AID (INT), name (VARCHAR)	PK: AID	Stores author names.
Book_Authors	AID (INT), ISBNID (VARCHAR)	PK: (AID, ISBNID), FK: AID, ISBNID	Resolves the M:N relationship between Books and Authors.
Publisher_Order	order_id (INT), ISBNID (VARCHAR), PID (INT), order_date (DATE), quantity (INT), status (ENUM)	PK: order_id, FK: ISBNID, PID	Tracks replenishment orders. Status: 'Pending', 'Confirmed'.
Sales	sale_id (INT), user_id (INT), ISBNID (VARCHAR),	PK: sale_id, FK: user_id, ISBNID	Records customer purchase

Table Name	Attributes	Constraints	Description
	sale_date (DATE), quantity (INT), price_at_sale (DECIMAL)		transactions.
Cart	cart_id (INT), customer_id (INT), ISBNID (VARCHAR), quantity (INT)	PK: cart_id, FK: customer_id, ISBNID, Unique: (customer_id, ISBNID)	Manages temporary shopping cart items for customers.

2. Database Integrity and Automation (Triggers)

To ensure the system remains consistent without manual intervention, we implemented three critical triggers directly in the database:

1. **Prevent Negative Stock (`prevent_negative_stock`):** A `BEFORE UPDATE` trigger on the **Book** table. It checks if a sale would result in `quantity < 0` and raises a signal (`SQLSTATE '45000'`) to block the transaction if so.
2. **Automated Reordering (`auto_reorder_book`):** An `AFTER UPDATE` trigger on the **Book** table. When a sale causes the `quantity` to drop below the `threshold`, it automatically inserts a 'Pending' order into the **Publisher_Order** table with a default quantity of 20.
3. **Stock Replenishment (`confirm_order_add_stock`):** A `BEFORE UPDATE` trigger on the **Publisher_Order** table. When the status changes from 'Pending' to 'Confirmed', it automatically updates the **Book** table to add the ordered quantity to the current stock.

3. Implemented Features and UI Logic

The system supports two distinct user roles: **Administrators** and **Customers**. The logic for each interface is designed to interact seamlessly with the database schema and enforced integrity rules.

3.1 Customer Interface Logic

Feature	UI Screen Logic	Database Implementation
Search for Books	Users can search by ISBN, Title, Category, or Author. The UI displays book details and current stock.	Executes `SELECT` queries with `JOIN` on **Book**, **Book_Authors**, and **Author** tables.
Shopping Cart	Allows adding books to a cart, viewing current items, and updating quantities.	**Add/Update:** Inserts or updates records in the **Cart** table. A `UNIQUE` constraint on `(customer_id, ISBNID)` prevents duplicate entries.
Checkout	Finalizes the purchase by collecting payment info and creating a sale record.	**Transaction:** 1) Inserts a record into **Sales**. 2) **Updates** `Book.quantity` (deducting the sold amount). 3) Clears the **Cart** for that user.
View Past Orders	Displays a history of all purchases made by the customer.	Queries the **Sales** table filtered by the current `user_id`.

3.2 Administrator Interface Logic

Feature	UI Screen Logic	Database Implementation
Add/Modify Books	Admin forms for adding new titles or updating existing stock levels and prices.	Direct `INSERT` or `UPDATE` on the **Book** table. **Trigger 1** ensures stock never goes negative.
Place Orders	Admin can manually place replenishment orders or view automated ones.	Inserts into **Publisher_Order**. **Trigger 2** automatically creates these when stock hits the `threshold`.
Confirm Orders	Admin confirms receipt of books from a publisher.	Updates `Publisher_Order.status` to 'Confirmed'. **Trigger 3** then automatically increases `Book.quantity`.
System Reports	Dashboard for generating sales and inventory reports.	Uses stored procedures to calculate total sales, top customers, and best-selling books.

4. Conclusion

The **Order Processing System** successfully implements a full-stack database solution for an online bookstore. By utilizing advanced SQL features like triggers and ENUM types, and by strictly adhering to a normalized schema, we have created a system that is both functional for users and robust against data inconsistency. The implementation fully meets the requirements for Alexandria University's Database Systems course.